

MI SCL API

Version 3.4

©2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020
3.1	Added Muffin flow chart	07/25/2021
	Added PROCFS introduction	08/25/2021
3.2	Added Mochi info	02/17/2022
3.3	- Added MI_SCL_StretchBufOsd Info - Added PROCFS debug cmd	03/21/2022
3.4	Added Maruko information	03/22/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. 概述.....	1
1.1. 模块说明.....	1
1.2. 流程框图.....	1
1.2.1 Tiramisu 框图.....	1
1.2.2 Muffin 框图.....	4
1.2.3 Mochi 框图.....	7
1.2.4 Maruko 框图.....	9
1.3. 关键字说明.....	11
2. API 参考.....	12
2.1. MI_SCL_CreateDevice.....	12
2.2. MI_SCL_DestroyDevice.....	15
2.3. MI_SCL_CreateChannel.....	16
2.4. MI_SCL_DestroyChannel.....	17
2.5. MI_SCL_SetChnParam.....	17
2.6. MI_SCL_GetChnParam.....	18
2.7. MI_SCL_SetInputPortCrop.....	19
2.8. MI_SCL_GetInputPortCrop.....	20
2.9. MI_SCL_StartChannel.....	21
2.10. MI_SCL_StopChannel.....	22
2.11. MI_SCL_SetOutputPortParam.....	22
2.12. MI_SCL_GetOutputPortParam.....	23
2.13. MI_SCL_EnableOutputPort.....	24
2.14. MI_SCL_DisableOutputPort.....	25
2.15. MI_SCL_StretchBuf.....	26
2.16. MI_SCL_StretchBufOsd.....	27
3. SCL 数据类型.....	29
3.1. MI_SCL_DEV.....	29
3.2. MI_SCL_CHANNEL.....	30
3.3. MI_SCL_PORT.....	30
3.4. MI_SCL_HWScId_e.....	30
3.5. MI_SCL_DevAttr_t.....	31
3.6. MI_SCL_ChannelAttr_t.....	31
3.7. MI_SCL_ChnParam_t.....	32
3.8. MI_SCL_OutPortParam_t.....	32
3.9. MI_SCL_FilterType_e.....	33
3.10. MI_SCL_DirectBuf_t.....	34
3.11. MI_SCL_DirectOsdArgb1555Alpha_t.....	35
3.12. MI_SCL_DirectOsdAlphaModePara_u.....	35
3.13. MI_SCL_DirectOsdAlphaMode_e.....	36
3.14. MI_SCL_DirectOsdAlphaAttr_t.....	36
3.15. MI_SCL_DirectOsdInfo_t.....	37

3.16. MI_SCL_DirectOsdBuf_t.....	38
4. SCL 错误码.....	39
5. PROCFS 介绍.....	40
5.1. cat.....	40
5.2. echo.....	42

1. 概述

1.1. 模块说明

SCL 将原始画面缩放到各个 output port 指定分辨率。每个 scl 按照输入数据源的类型划分成不同的 Device，每一个 output port 端口有 crop/scaling/mirror/flip/pixel 转换功能。

1.2. 流程框图

1.2.1 Tiramisu 框图

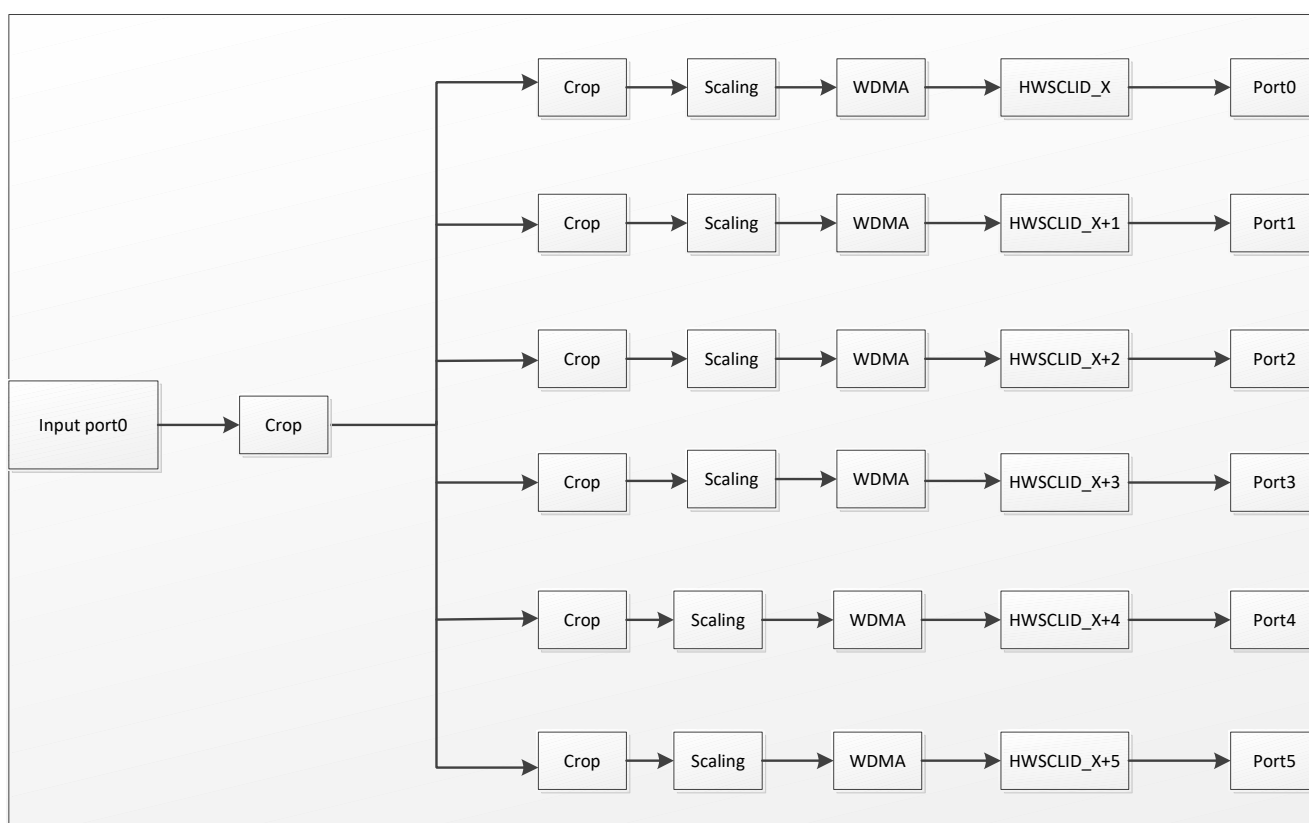


图 1-1: Tiramisu 框图

Tiramisu MI_SCL 总共有 6 个 scl 硬件，按照输入源的类型，划分成不同的 Device。

Device Id	Input port 输入源类型
Device0	只支持和 MI_ISP output port0 realtime 连接
Device1	支持从 Dram 读取数据，支持 MI_JPD realtime 绑定
Device2	只支持和 MI_VIF YUV Realtime 连接
Device3	只支持从 Dram 读取数据，只可以使用 E_MI_SCL_HWSCL5, 支持 Rotation, 做 rotation 的时候功能限制见 MI_SCL_SetChnParam

Tiramisu Input 规格

Device Id	非 yuv420 Max width	Yuv420 Max width	Pixel format	Bind type
0	/	/	YUV422	Realtime Bind ISP
1	8192	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.Frame mode 2.Realtime Bind JPD
2	/	/	YUV422	Realtime Bind VIF
3	8192	8192	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	Frame mode

Tiramisu HWSclId Output 规格

HWSclId	非 yuv420 Max width	Yuv420 Max width	Pixel format	Bind type
0	8192	4608	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.Frame mode 2.Realtime Bind Venc(JPEG) 3.Ring/HW_AUTOSYNC Bind Venc(h26x)
1	8192	4608	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
2	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
3	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
4	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
5	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/	1.同 Port0 2.支持 disp rot 时使用

HWScId	非 yuv420 Max width	Yuv420 Max width	Pixel format	Bind type
			BGRA8888	

1.2.2 Muffin 框图

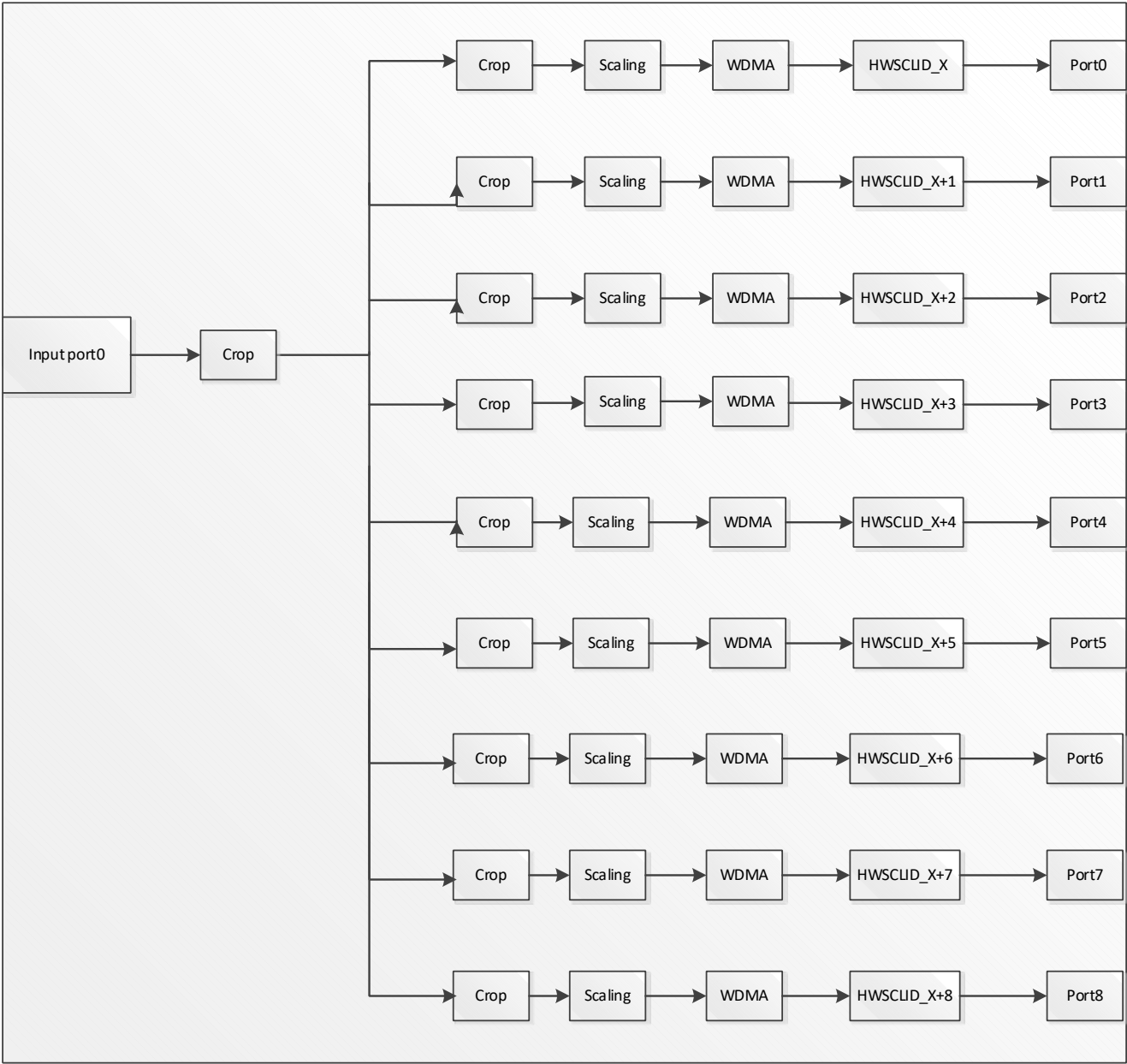


图 1-2: Muffin 框图

MI_SCL muffin 总共有 9 个 scl 硬件， 按照输入源的类型， 划分成不同的 Device。

Device Id	Input port 输入源类型
Device0	只支持和 MI_ISP device0 output port0 realtime 连接
Device1	支持从 Dram 读取数据， 支持 MI_JPD realtime 绑定
Device2	只支持和 MI_VIF device8 YUV Realtime 连接
Device3	只支持从 Dram 读取数据， 只可以使用 E_MI_SCL_HWSCL8, 支持 Rotation, 做 rotation 的时候功能限制见 MI_SCL_SetChnParam

Device Id	Input port 输入源类型
Device4	只支持和 MI_ISP device1 output port0 realtime 连接
Device5	支持从 Dram 读取数据，支持 MI_JPD realtime 绑定
Device6	只支持和 MI_VIF device24 YUV Realtime 连接

Muffin Input 规格

Device Id	非 yuv420 Max width	Yuv420 Max width	Pixel format	Bind type
Device0	/	/	YUV422	Realtime Bind ISP
Device1	8192	7680	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1. Frame mode 2. Realtime Bind JPD
Device2	/	/	YUV422	Realtime Bind VIF
Device3	8192	7680	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	Frame mode
Device4	/	/	YUV422	Realtime Bind ISP
Device5	8192	8192	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1. Frame mode 2. Realtime Bind JPD
Device6	/	/	YUV422	Realtime Bind VIF

Muffin HWSclId Output 规格

HWSclId	非 yuv420 Max width	Yuv420 Max width	Pixel format	Bind type
0	8192	7680	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.Frame mode 2.Ring Bind Venc(h26x/JPEG)
1	8192	7680	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
2	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.同 Port0 2.支持 disp rot 时使用

HWSclId	非 yuv420 Max width	Yuv420 Max width	Pixel format	Bind type
3	1920	1920	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
4	8192	7680	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
5	8192	7680	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
6	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.同 Port0 2.支持 disp rot 时使用
7	1920	1920	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
8	8192	8192	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.同 Port0 2.支持 disp rot 时使用

※ 注意

- 相同的 HWSclId 不可以跨 Device 使用。
例如：
Dev0 的 pstSclDevAttr->u32NeedUseHWOutPortMask= E_MI_SCL_HWSCL2|
E_MI_SCL_HWSCL3;
Dev1 的 pstSclDevAttr->u32NeedUseHWOutPortMask= E_MI_SCL_HWSCL0|
E_MI_SCL_HWSCL2;
这样 HWSCL2 就被 Dev0/Dev1 同时使用，创建 Device 时会返回 err 状态。
- output portid 按照 MI_SCL_CreateDevice 时设置 u32NeedUseHWOutPortMask 从低位到高位依次和 HWSCLID 映射。
- output port 和后端 realtime/ring/hw_autosync 绑定时，后端仅支持和一个 output port id 连接。可以支持多个 channel 但是同一个 output port id 绑定 realtime 或者 hw_autosync(h26x)。

1.2.3 Mochi 框图

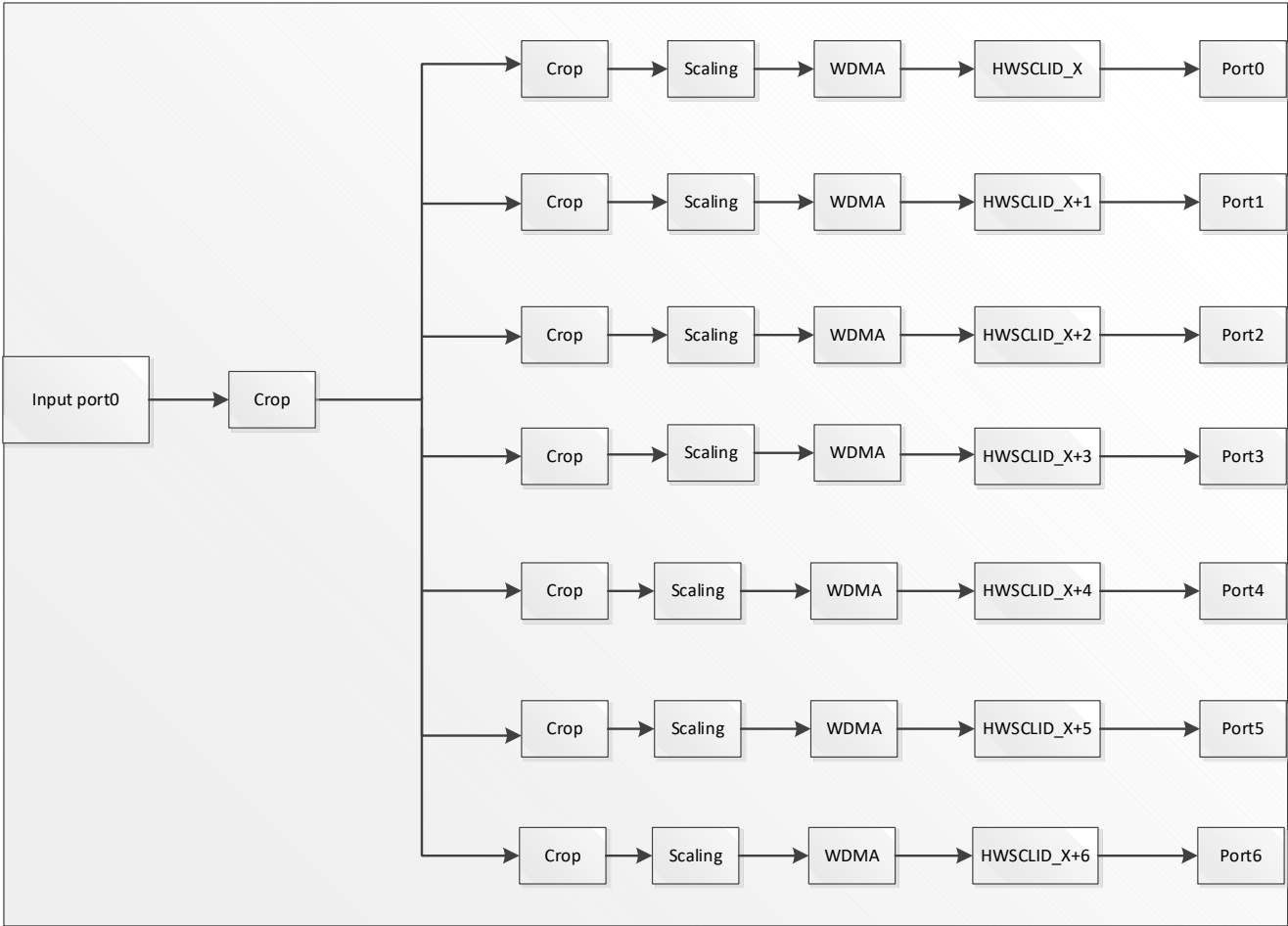


图 1-3: Mochi 框图

Mochi MI_SCL 总共有 7 个 scl 硬件，按照输入源的类型，划分成不同的 Device。

Device Id	Input port 输入源类型
Device0	只支持和 MI_ISP output port0 realtime 连接
Device1	支持从 Dram 读取数据，支持 MI_JPD realtime 绑定

Mochi Input 规格

Device Id	非 YUV420 Max width	YUV420 Max width	Pixel format	Bind type
0	/	/	YUV422	Realtime Bind ISP
1	8192	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.Frame mode 2.Realtime Bind JPD

Mochi HWSclId Output 规格

HWSclId	非 yuv420 Max width	Yuv420 Max width	Pixel format	Bind type
0	8192	4096	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.Frame mode 2.Realtime Bind Venc(JPEG) 3.Ring Bind Venc(h26x/ JPEG)
1	8192	4096	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
2	8192	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
3	3840	1920	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
4	3840	1280	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
5	3840	1280	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
6	8192	4096	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0

1.2.4 Maruko 框图

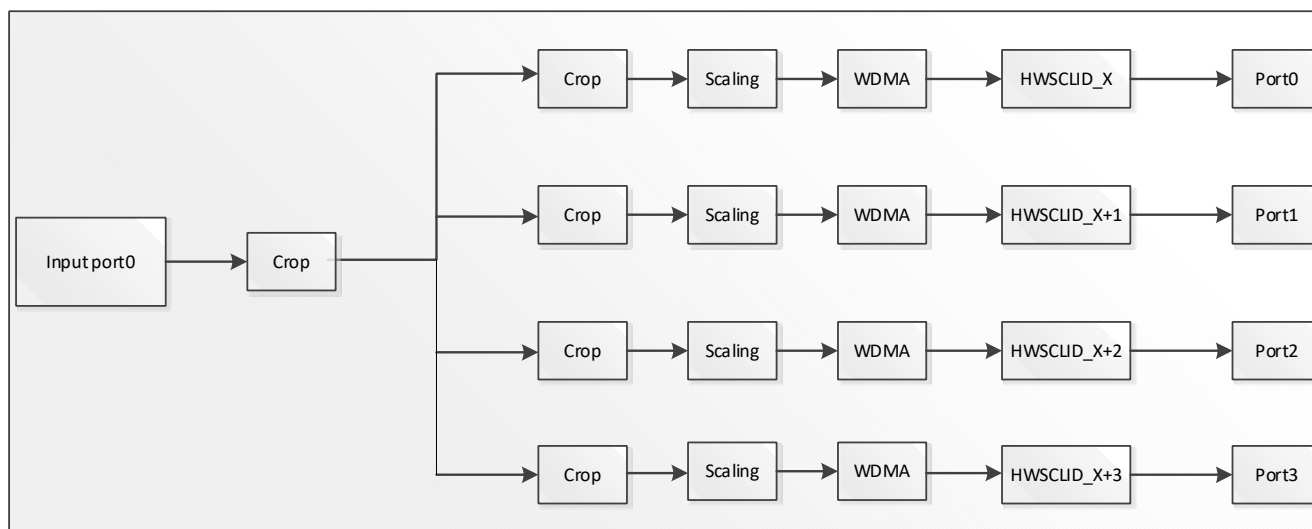


图 1-4: Maruko 框图

Maruko MI_SCL 总共有 4 个 scl 硬件，按照输入源的类型，划分成不同的 Device。

Device Id	Input port 输入源类型
Device0	只支持和 MI_ISP output port0 realtime 连接
Device1	支持从 Dram 读取数据
Device2	只支持和 MI_VIF output port0 yuv 格式 realtime 连接。

Maruko Input 规格

Device Id	非 yuv420 Max width	YUV420 Max width	Pixel format	Bind type
0	/	/	YUV422	Realtime Bind ISP
1	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	Frame mode
2	/	/	YUV422	Realtime Bind VIF

Maruko HWScId Output 规格

HWScId	非 yuv420 Max width	YUV420 Max width	Pixel format	Bind type
0	3840	3840	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	1.Frame mode 2.Realtime Bind Venc(JPEG) 3.Ring Bind Venc(h26x/ JPEG)
1	3840	3840	YUV420/YUV422/	同 Port0

HWSclId	非 yuv420 Max width	YUV420 Max width	Pixel format	Bind type
			ARGB8888/ ABGR8888/ BGRA8888	
2	1920	1920	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0
3	1920	1920	YUV420/YUV422/ ARGB8888/ ABGR8888/ BGRA8888	同 Port0

1.3. 关键字说明

- **Device**
Device 由固定的 input 类型，和用户分配的 HWSclId 组成。
- **Channel**
一个 Device 可以分时复用为多个通道， 每个通道为一个 channel。
- **Input Port**
Channel 上的输入端口，每个 Device 支持的输入类型请参考流程框图。
- **Output port**
Device 上的输出端口，输出端口和 Device 的 HWSclId 一一对应。

2. API 参考

该功能模块提供以下 API:

表 2-1: API 参考

API名	功能
MI_SCL_CreateDevice	创建SCL设备
MI_SCL_DestroyDevice	销毁SCL设备
MI_SCL_CreateChannel	创建SCL 通道
MI_SCL_DestroyChannel	销毁SCL 通道
MI_SCL_SetChnParam	设置通道属性
MI_SCL_GetChnParam	获取通道属性
MI_SCL_SetInputPortCrop	设置通道input 端口crop
MI_SCL_GetInputPortCrop	获取通道input端口crop 信息
MI_SCL_StartChannel	启动通道
MI_SCL_StopChannel	禁用通道
MI_SCL_SetOutputPortParam	设置output 端口参数
MI_SCL_GetOutputPortParam	获取output 端口参数
MI_SCL_EnableOutputPort	使能output 端口
MI_SCL_DisableOutputPort	禁用output 端口
MI_SCL_StretchBuf	缩放、拉伸、裁剪指定内存中的图像数据
MI_SCL_StretchBufOsd	在缩放、拉伸、裁剪指定内存中的图像数据基础上叠加OSD图像数据

2.1. MI_SCL_CreateDevice

➤ 功能

创建一个 SCL 设备。

➤ 语法

MI_S32 MI_SCL_CreateDevice(MI_SCL_DEV DevId, [MI_SCL_DevAttr_t](#) *pstSclDevAttr)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号。	输入
pstSclDevAttr	Scl 设备属性。	输入

➤ 返回值

返回值 {
MI_SUCCESS(0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

- 每个 Device 的特性参考 1.2. 章节中 注意说明。
- stCreateDevAttr.u32NeedUseHWOutPortMask = E_MI_SCL_HWSCL2| E_MI_SCL_HWSCL3| E_MI_SCL_HWSCL4;

每个channel的输出端口和Device上的HWSCL ID之间对应关系为：

Port0 → HWSCL2

Port1 → HWSCL3

Port2 → HWSCL4

Portid 与HWSCL bitmask 从低到高对应。

- Tiramisu 中，有 MI_VDEC 使用场景，Device3 会被内部创建使用，其它 Device 不能再使用 HWSCL5；用户创建 Device3 对应的 channel 使用。
- Muffin 中，有 MI_VDEC 使用场景，Device3 会被内部创建使用，其它 Device 不能再使用 HWSCL8；用户创建 Device3 对应的 channel 使用。
- Mochi 中，有 MI_VDEC 使用场景，Device1 会被内部创建使用，其它 Device 不能再使用 HWSCL6；用户创建 Device1 对应的 channel 使用。

➤ 举例

MI_SCL 初始化流程：

```
MI_S32 ST_SclModuleInit(MI\_SCL\_DEV SclDevId)
{
#define ST_MAX_ISP_OUTPORT_NUM (6) //Muffin (9), Tiramisu (6)
MI\_SCL\_CHANNEL SclChnId = 0;
MI\_SCL\_PORT SclOutPortId = 0;

MI\_SCL\_DevAttr\_t stCreateDevAttr;
memset(&stCreateDevAttr, 0x0, sizeof(MI_SCL_DevAttr_t));
stCreateDevAttr.u32NeedUseHWOutPortMask=E_MI_SCL_HWSCL2|E_MI_SCL_HWSCL3|
E_MI_SCL_HWSCL4;
MI\_SCL\_CreateDevice(SclDevId, &stCreateDevAttr);

MI\_SCL\_ChannelAttr\_t stSclChnAttr;
```

```

memset(&stSclChnAttr, 0x0, sizeof(MI_SCL_ChannelAttr_t));

MI\_SCL\_CreateChannel(SclDevId, SclChnId, &stSclChnAttr);

MI_SYS_WindowRect_t stCropInfo;
stCropInfo.u16x=0;
stCropInfo.u16y=0;
stCropInfo.u16Width=0;
stCropInfo.u16Height=0;//width/height use 0, not use crop
MI\_SCL\_SetInputPortCrop(SclDevId, SclChnId, &stCropInfo);

MI\_SCL\_ChnParam\_t stSclChnParam;
memset(&stSclChnParam, 0x0, sizeof(MI_SCL_ChnParam_t));
stSclChnParam.eRot = E_MI_SYS_ROTATE_NONE;
MI\_SCL\_SetChnParam(SclDevId, SclChnId, &stSclChnParam);

MI\_SCL\_StartChannel(SclDevId, SclChnId);

for(SclOutPortId=0; SclOutPortId<ST_MAX_SCL_OUTPORT_NUM; SclOutPortId++)
{
    MI\_SCL\_OutPortParam\_t stSclOutputParam;
    memset(&stSclOutputParam, 0x0, sizeof(MI_SCL_OutPortParam_t));
    stSclOutputParam.stSCLOutCropRect.u16x=0;
    stSclOutputParam.stSCLOutCropRect.u16y=0;
    stSclOutputParam.stSCLOutCropRect.u16Width =0;
    stSclOutputParam.stSCLOutCropRect.u16Height =0;
    //width/height use 0, not use crop
    stSclOutputParam.stSCLOutputSize.u16Width=1920;
    stSclOutputParam.stSCLOutputSize.u16Height=1080;
    stSclOutputParam.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
    stSclOutputParam.bMirror =FALSE;
    stSclOutputParam.bFlip =FALSE;
    MI\_SCL\_SetOutputPortParam(SclDevId, SclChnId, SclOutPortId,
&stSclOutputParam);
    MI\_SCL\_EnableOutputPort(SclDevId, SclChnId, SclOutPortId);
}

return MI_SUCCESS;
}

```

MI_SCL 去初始化流程:

```
MI_S32 ST_SclModuleUnInit(MI_SCL_DEV SclDevId)
{
    #define ST_MAX_SCL_OUTPORT_NUM (6) //Muffin (9), Tiramisu (6)
    MI_SCL_CHANNEL SclChnId = 0;
    MI_SCL_PORT SclOutPortId = 0;

    for(SclOutPortId=0; SclOutPortId<ST_MAX_SCL_OUTPORT_NUM; SclOutPortId++)
    {
        MI\_SCL\_DisableOutputPort(SclDevId, SclChnId, SclOutPortId);
    }

    MI\_SCL\_StopChannel(SclDevId, SclChnId);
    MI\_SCL\_DestroyChannel(SclDevId, SclChnId);

    MI\_SCL\_DestroyDevice(SclDevId);

    return MI_SUCCESS;
}
```

2.2. MI_SCL_DestroyDevice

➤ 功能

销毁一个 SCL 设备。

➤ 语法

MI_S32 MI_SCL_DestroyDevice(MI_SCL_DEV DevId)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入

➤ 返回值

返回值 {
MI_SUCCESS(0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

➤ 注意

需要先禁用掉设备上所有的输出端口和通道，否则会返回失败。

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_CreateDevice](#)

2.3. MI_SCL_CreateChannel

➤ 功能

设备上创建一个通道。

➤ 语法

MI_S32 MI_SCL_CreateChannel(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, [MI_SCL_ChannelAttr_t](#) *pstChnAttr)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号，取值范围(0~31)。	输入
pstChnAttr	SCL 通道属性	输入

➤ 返回值

返回值 {
MI_SUCCESS(0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

需要先创建设备才能创建通道。

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_DestroyChannel](#)

2.4. MI_SCL_DestroyChannel

➤ 功能

销毁设备上的一个通道。

➤ 语法

MI_S32 MI_SCL_DestroyChannel(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入

➤ 返回值

错误码。' (Failure)." data-bbox="145 363 215 414"/>
 返回值
 MI_SUCCESS(0) 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

需要关掉通道上的所有输出端口，才能销毁通道。

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_CreateChannel](#)

2.5. MI_SCL_SetChnParam

➤ 功能

动态设置通道参数。

➤ 语法

MI_S32 MI_SCL_SetChnParam(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, [MI_SCL_ChnParam_t](#) *pstChnParam)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入
pstChnParam	通道参数	输入

➤ 返回值

返回值 {
MI_SUCCESS(0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

➤ 注意

- 通道已经在运行的情况下，需要改变通道参数，需要先使用 MI_SCL_StopChannel 将通道停下，清除缓存数据。
- rotation 90、180、270 时，output 端口仅支持 rotation 功能，不支持 crop/scaling/pixel format 转换/Mirror/Flip。
- rotation 90、180、270 时，不支持 MI_RGN attach 叠加 osd/cover。
- 由于 rotation 不支持 scaling，output width 需要 8 对齐，所以 rot 90/270 的 input height 需要 8 对齐，否则内部检查会报错。
- 芯片支持 Rotation 的情况：

芯片	支持旋转的设备号
Tiramisu	Device3
Muffin	Device3
Mochi	不支持旋转
Maruko	不支持旋转

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_GetChnParam](#)

2.6. MI_SCL_GetChnParam

➤ 功能

获取通道参数。

➤ 语法

MI_S32 MI_SCL_GetChnParam(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, [MI_SCL_ChnParam_t](#) *pstChnParam)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入
pstChnParam	SCL 通道参数	输出

➤ 返回值

返回值	MI_SUCCESS(0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

在没有设置的情况下，获取到的参数都为 0。

➤ 举例

无。

➤ 相关主题

[MI_SCL_SetChnParam](#)

2.7. MI_SCL_SetInputPortCrop

➤ 功能

在输入端口设置裁剪区域。

➤ 语法

MI_S32 MI_SCL_SetInputPortCrop(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, MI_SYS_WindowRect_t *pstCropInfo)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入
pstCropInfo	裁剪区域	输入

➤ 返回值

返回值 {
MI_SUCCESS(0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

- 只有输入端口是从 DRAM 读取数据时，该设置才能生效。
- 当输入数据压缩模式不为 E_MI_SYS_COMPRESS_MODE_NONE 时，输入数据不支持裁剪。

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_GetInputPortCrop](#)

2.8. MI_SCL_GetInputPortCrop

➤ 功能

获取通道输入端口裁剪区域。

➤ 语法

MI_S32 MI_SCL_GetInputPortCrop(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, MI_SYS_WindowRect_t *pstCropInfo)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入
pstCropInfo	裁剪区域	输出

➤ 返回值

返回值 {
MI_SUCCESS(0) 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件: `mi_scl_datatype.h`、`mi_scl.h`
 - 库文件:
- ※ 注意

当不需要裁剪时, 获取到的是输入端口分辨率
- 举例

无。
- 相关主题

[MI_SCL_SetInputPortCrop](#)

2.9. MI_SCL_StartChannel

- 功能

启用设备上的一个通道。
- 语法

MI_S32 MI_SCL_StartChannel(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId)

- 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入

- 返回值

{

返回值

MI_SUCCESS(0) 成功。

非 0 失败, 详情参照[错误码](#)。

- 依赖
 - 头文件: `mi_scl_datatype.h`、`mi_scl.h`
 - 库文件:

- ※ 注意

和 [MI_SCL_StopChannel](#) 成对调用。

- 举例

参考 [MI_SCL_CreateDevice](#) 举例。

- 相关主题

[MI_SCL_StopChannel](#)

2.10. MI_SCL_StopChannel

➤ 功能

停用设备上的一个通道。

➤ 语法

MI_S32 MI_SCL_StopChannel(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId)

➤ 形参

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入

➤ 返回值

返回值	MI_SUCCESS(0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

在通道创建的情况下，都可以停下通道，通道停下所有的输出端口都不会有数据输出。

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_StartChannel](#)

2.11. MI_SCL_SetOutputPortParam

➤ 描述

设置通道输出端口参数。

➤ 语法

MI_S32 MI_SCL_SetOutputPortParam(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, [MI_SCL_PORT](#) PortId, [MI_SCL_OutPortParam_t](#) *pstOutPortParam)

➤ 参数

参数名称	描述	输入/输出
DevId	SCL 设备号	输入

ChnId	SCL 通道号	输入
PortId	输出端口号	输入
pstOutPortParam	输出端口参数	输入

➤ 返回值

返回值 { MI_SUCCESS(0)成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

- 在输出端口已经启用的情况下，需要改变输出参数，需要先调用 MI_SCL_DisableOutputPort 禁用端口，再来设置参数，然后再启用输出端口。
- 当 VIF->ISP->SCL 均为 Realtime 绑定时，SCL Output 设置 Scaling up 存在 FIFO FULL 风险，不建议设置 Scaling up。

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_GetOutputPortParam](#)

2.12. MI_SCL_GetOutputPortParam

➤ 描述

获取输出端口参数。

➤ 语法

MI_S32 MI_SCL_GetOutputPortParam(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, [MI_SCL_PORT](#) PortId, [MI_SCL_OutPortParam_t](#) *pstOutPortParam)

➤ 参数

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入
PortId	SCL 输出端口号	输入
pstOutPortParam	SCL 输出端口参数	输出

➤ 返回值

返回值 { MI_SUCCESS(0)成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_scl_datatype.h、mi_scl.h
 - 库文件：

※ 注意
无。

➤ 举例
无。

➤ 相关主题
[MI_SCL_SetOutputPortParam](#)

2.13. MI_SCL_EnableOutputPort

➤ 描述
使能输出端口。

➤ 语法
MI_S32 MI_SCL_EnableOutputPort(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, [MI_SCL_PORT](#) PortId)

➤ 参数

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入
PortId	SCL 输出端口号	输入

➤ 返回值

返回值 {

- MI_SUCCESS(0)成功。
- 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意
和 [MI_SCL_DisableOutputPort](#) 成对调用。

➤ 举例
参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_DisableOutputPort](#)

2.14. MI_SCL_DisableOutputPort

➤ 描述

禁用输出端口。

➤ 语法

MI_S32 MI_SCL_DisableOutputPort(MI_SCL_DEV DevId, [MI_SCL_CHANNEL](#) ChnId, [MI_SCL_PORT](#) PortId)

➤ 参数

参数名称	描述	输入/输出
DevId	SCL 设备号	输入
ChnId	SCL 通道号	输入
PortId	输出端口号	输入

➤ 返回值

返回值 { MI_SUCCESS(0)成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

和 [MI_SCL_EnableOutputPort](#) 成对调用。

➤ 举例

参考 [MI_SCL_CreateDevice](#) 举例。

➤ 相关主题

[MI_SCL_EnableOutputPort](#)

2.15. MI_SCL_StretchBuf

➤ 描述

缩放、拉伸、裁剪指定内存中的图像数据。

➤ 语法

MI_S32 MI_SCL_StretchBuf([MI_SCL_DirectBuf_t](#) *pstSrcBuf, MI_SYS_WindowRect_t *pstSrcCrop, [MI_SCL_DirectBuf_t](#) *pstDstBuf, [MI_SCL_FilterType_e](#) eFilterType)

➤ 参数

参数名称	描述	输入/输出
pstSrcBuf	源数据buffer参数指针	输入
pstSrcCrop	源数据裁剪区域	输入
pstDstBuf	输出buffer参数指针	输出
eFilterType	Scl 使用滤波器类型	输入

➤ 返回值

返回值 { MI_SUCCESS(0)成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_scl_datatype.h、mi_scl.h
- 库文件：

※ 注意

- 需要提前创建对应的 DEV。
- 该接口是阻塞形式，这帧数据做完才会结束。

不同系列芯片对应使用的 DEVID:

芯片	StretchBuf 使用的 DEVID
Tiramisu	Device3
Muffin	Device3
Mochi	Device1
Maruko	Device1

输入宽、高的对齐要求:

pixel format	width	height
YUV420SPNV12, YUV420SPNV21	16	2
YUV422YUYV, YUV422UYVY, YUV422YVYU, YUV422VYUY	8	2
ARGB8888, ABGR8888, BGRA8888	4	2

输出宽、高的对齐要求:

pixel format	width	height
YUV420SPNV12, YUV420SPNV21	8	2
YUV422YUYV, YUV422UYVY, YUV422YVYU, YUV422VYUY	8	2
ARGB8888, ABGR8888, BGRA8888	8	2

➤ 举例

无。

➤ 相关主题

[MI_SCL_CreateDevice](#)

2.16. MI_SCL_StretchBufOsd

➤ 描述

在缩放、拉伸、裁剪指定内存中的图像数据基础上叠加 OSD 图像数据。

➤ 语法

```
MI_S32 MI_SCL_StretchBufOsd(MI_SCL_DirectBuf_t *pstSrcBuf,
MI_SYS_WindowRect_t*pstSrcCrop,MI_SCL_DirectBuf_t *pstDstBuf,
MI_SCL_DirectOsdBuf_t*pstDstOsdBuf,MI_SCL_FilterType_e eFilterType);
```

➤ 参数

参数名称	描述	输入/输出
pstSrcBuf	源数据buffer参数指针	输入
pstSrcCrop	源数据裁剪区域	输入
pstDstBuf	输出buffer参数指针	输出
pstDstOsdBuf	叠加在pstDstBuf的Osdbuffer参数指针	输入
eFilterType	Scl 使用滤波器类型	输入

➤ 返回值

返回值 { MI_SUCCESS(0)成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件: mi_scl_datatype.h、mi_scl.h
- 库文件:

※ 注意

- 需要提前创建对应的 DEV。
- 该接口是阻塞形式，这帧数据做完才会结束。

不同系列芯片对应使用的 DEVID:

芯片	StretchBuf 使用的 DEVID
Tiramisu	Device3
Muffin	Device3
Mochi	Device1
Maruko	Device1

输入宽、高的对齐要求:

pixel format	width	height
YUV420SPNV12, YUV420SPNV21	16	2
YUV422YUYV, YUV422UYVY, YUV422YVYU, YUV422VYUY	8	2
ARGB8888, ABGR8888, BGRA8888	4	2

输出宽、高的对齐要求:

pixel format	width	height
YUV420SPNV12, YUV420SPNV21	8	2
YUV422YUYV, YUV422UYVY, YUV422YVYU, YUV422VYUY	8	2
ARGB8888, ABGR8888, BGRA8888	8	2

➤ 举例

无。

➤ 相关主题

[MI_SCL_CreateDevice](#)

3. SCL 数据类型

视频输入相关数据类型定义如下：

表 3-2: SCL 数据类型

数据类型	描述
MI_SCL_DEV	SCL 设备ID
MI_SCL_CHANNEL	SCL 通道ID
MI_SCL_PORT	SCL 输出端口ID
MI_SCL_HWSclId_e	SCL 输出端口对应硬件scaling ID
MI_SCL_DevAttr_t	SCL 设备属性
MI_SCL_ChannelAttr_t	SCL 通道静态属性
MI_SCL_ChnParam_t	SCL 通道动态属性
MI_SCL_OutPortParam_t	SCL 输出端口参数
MI_SCL_FilterType_e	SCL 滤波类型
MI_SCL_DirectBuf_t	SCL操作指定buffer 参数类型
MI_SCL_DirectOsdArgb1555Alpha_t	Argb1555格式的前景、背景Alpha设定
MI_SCL_DirectOsdAlphaModePara_u	定义Osd Alpha Mode参数联合体
MI_SCL_DirectOsdAlphaMode_e	设定Osd Alpha显示模式
MI_SCL_DirectOsdAlphaAttr_t	定义Osd Alpha属性的结构体
MI_SCL_DirectOsdInfo_t	SCL 直接操作Osdbuffer 参数信息
MI_SCL_DirectOsdBuf_t	SCL 直接操作 Osdbuffer 参数类型。

3.1. MI_SCL_DEV

➤ 说明

定义 SCL 设备 ID 类型。

➤ 定义

```
typedef MI_U32 MI_SCL_DEV;
```

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.2. MI_SCL_CHANNEL

- 说明
定义 SCL 通道 ID 类型。
- 定义

```
typedef MI_U32 MI_SCL_CHANNEL;
```
- ※ 注意事项
无。
- 相关数据类型及接口
无。

3.3. MI_SCL_PORT

- 说明
定义 SCL 输出端口 ID 类型。
- 定义

```
typedef MI_U32 MI_SCL_PORT;
```
- ※ 注意事项
无。
- 相关数据类型及接口
无。

3.4. MI_SCL_HWSclId_e

- 说明
SCL 设备使用的硬件 scaler Id 枚举。
- 定义

```
typedef enum  
{  
    E_MI_SCL_HWSCLID_INVALID= 0,  
    E_MI_SCL_HWSCL0      = 0x0001,  
    E_MI_SCL_HWSCL1      = 0x0002,  
    E_MI_SCL_HWSCL2      = 0x0004,  
    E_MI_SCL_HWSCL3      = 0x0008,  
    E_MI_SCL_HWSCL4      = 0x0010,  
    E_MI_SCL_HWSCL5      = 0x0020,  
    E_MI_SCL_HWSCL6      = 0x0040,  
    E_MI_SCL_HWSCL7      = 0x0080,  
    E_MI_SCL_HWSCL8      = 0x0100,  
    E_MI_SCL_HWSCL_MAX   = 0xffff,  
}MI_SCL_HWSclId_e;
```

※ 注意事项

Tiramisu 支持 E_MI_SCL_HWSCL0~E_MI_SCL_HWSCL5。
Muffin 支持 MI_SCL_HWSCL0~E_MI_SCL_HWSCL8。
Mochi 支持 MI_SCL_HWSCL0~E_MI_SCL_HWSCL6。
Maruko 支持 MI_SCL_HWSCL0~E_MI_SCL_HWSCL3

➤ 相关数据类型及接口

[MI_SCL_DevAttr_t](#)

3.5. MI_SCL_DevAttr_t

➤ 说明

SCL 设备属性。

➤ 定义

```
typedef struct MI_SCL_DevAttr_s
{
    MI_U32 u32NeedUseHWOutPortMask;
}MI_SCL_DevAttr_t;
```

➤ 成员

成员名称	描述
u32NeedUseHWOutPortMask	Device需要使用的Scaler id。

※ 注意事项

- 由 MI_SCL_HWSclId_e 成员 bit mask 组成。
- 每个 HW scl id 特点参考 1.2 章节，HW sclid 规格，按照场景需求选择。

➤ 相关数据类型及接口

[MI_SCL_CreateDevice](#)

3.6. MI_SCL_ChannelAttr_t

➤ 说明

Scl 通道属性。

➤ 定义

```
typedef struct MI_SCL_ChannelAttr_s
{
    MI_U32 u32Reserved;
}MI_SCL_ChannelAttr_t;
```

➤ 成员

成员名称	描述
------	----

u32Reserved	预留参数。
-------------	-------

- ※ 注意事项
可直接设置 0。

- 相关数据类型及接口
[MI_SCL_CreateChannel](#)

3.7. MI_SCL_ChnParam_t

- 说明
SCL 通道动态参数类型

- 定义

```
typedef struct MI_SCL_ChnParam_s
{
    MI_SYS_Rotate_e eRot;
}MI_SCL_ChnParam_t;
```

- 成员

成员名称	描述
eRot	Rotation 参数

- ※ 注意事项
Mochi/Maruko 不支持 rotation，只能设置 E_MI_SYS_ROTATE_NONE。

- 相关数据类型及接口
[MI_SCL_SetChnParam](#)

3.8. MI_SCL_OutPortParam_t

- 说明
SCL 输出端口参数。

- 定义

```
typedef struct MI_SCL_OutPortParam_s
{
    MI_SYS_WindowRect_t stSCLOutCropRect;
    MI_SYS_WindowSize_t stSCLOutputSize;
    MI_BOOL bMirror;
    MI_BOOL bFlip;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;
}MI_SCL_OutPortParam_t;
```

➤ 成员

成员名称	描述
stSCLOutCropRect	输出裁剪区域
stSCLOutputSize	输出 size
bMirror	使能水平翻转
bFlip	使能垂直翻转
ePixelFormat	输出像素格式
eCompressMode	输出压缩格式

※ 注意事项

- eCompressMode 支持范围

芯片	支持的输出像素压缩模式
Tiramisu	E_MI_SYS_COMPRESS_MODE_NONE
Muffin	E_MI_SYS_COMPRESS_MODE_NONE
Mochi	E_MI_SYS_COMPRESS_MODE_NONE E_MI_SYS_COMPRESS_MODE_TO_6BIT
Maruko	E_MI_SYS_COMPRESS_MODE_NONE E_MI_SYS_COMPRESS_MODE_TO_6BIT E_MI_SYS_COMPRESS_MODE_AFBC

E_MI_SYS_COMPRESS_MODE_AFBC 同时只支持一个 HW SclId 使用。

➤ 相关数据类型及接口

[MI_SCL_SetOutputPortParam](#)

3.9. MI_SCL_FilterType_e

➤ 说明

Scl 滤波器类型。

➤ 定义

```
typedef enum
{
    E_MI_SCL_FILTER_TYPE_AUTO,
    E_MI_SCL_FILTER_TYPE_BYPASS,
    E_MI_SCL_FILTER_TYPE_BILINEAR,
    E_MI_SCL_FILTER_TYPE_BICUBIC,
    E_MI_SCL_FILTER_TYPE_MAX,
} MI_SCL_FilterType_e;
```

➤ 成员

成员名称	描述
E_MI_SCL_FILTER_TYPE_AUTO	自动选择滤波器

E_MI_SCL_FILTER_TYPE_BYPASS	不经过算法滤波器
E_MI_SCL_FILTER_TYPE_BILINEAR	双线性滤波器
E_MI_SCL_FILTER_TYPE_BICUBIC	双立方插值滤波器
E_MI_SCL_FILTER_TYPE_MAX	滤波器类型最大值

※ 注意事项

当选择和算法相同的滤波器类型，可以得到更加精准的算法结果。

➤ 相关数据类型及接口

[MI_SCL_StretchBuf](#)

3.10. MI_SCL_DirectBuf_t

➤ 说明

SCL 直接操作 **buffer** 参数类型。

➤ 定义

```
typedef struct MI_SCL_DirectBuf_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U32 u32Width;
    MI_U32 u32Height;
    MI_PHY phyAddr[2];
    MI_U32 u32Stride[2];
    MI_U32 u32BuffSize;
}MI_SCL_DirectBuf_t;
```

➤ 成员

成员名称	描述
ePixelFormat	Buffer 像素格式
u32Width	有效像素宽度
u32Height	有效像素高度
phyAddr	Buffer 物理地址
u32Stride	Buffer 一行占字节数
u32BuffSize	Buffer 占用空间

※ 注意事项

无

➤ 相关数据类型及接口

[MI_SCL_StretchBuf](#)

[MI_SCL_StretchBufOsd](#)

3.11. MI_SCL_DirectOsdArgb1555Alpha_t

➤ 说明

Argb1555 格式的前景、背景 Alpha 设定。

➤ 定义

```
typedef struct MI_SCL_DirectBufOsdArgb1555Alpha_s
{
    MI_U8 u8BgAlpha;
    MI_U8 u8FgAlpha;
} MI_SCL_DirectOsdArgb1555Alpha_t;
```

➤ 成员

成员名称	描述
u8BgAlpha	背景Alpha，Alpha bit为0时对应的Alpha值，取值范围0~0xFF。
u8FgAlpha	前景Alpha，Alpha bit为1时对应的Alpha值，取值范围0~0xFF。

※ 注意事项

无

➤ 相关数据类型及接口

[MI_SCL_StretchBufOsd](#)

3.12. MI_SCL_DirectOsdAlphaModePara_u

➤ 说明

定义 Osd Alpha Mode 参数联合体。

➤ 定义

```
typedef union
{
    MI_SCL_DirectOsdArgb1555Alpha_t stArgb1555Alpha;
    MI_U8 u8ConstantAlpha;
} MI_SCL_DirectOsdAlphaModePara_u;
```

➤ 成员

成员名称	描述
stArgb1555Alpha	Pixelalpha时Argb1555格式的前景Alpha及背景Alpha设定
u8ConstantAlpha	Constant Alpha时Alpha值设定，取值范围0~0xFF

※ 注意事项

无

- 相关数据类型及接口

[MI_SCL_StretchBufOsd](#)

3.13. MI_SCL_DirectOsdAlphaMode_e

- 说明

设定 Osd Alpha 显示模式。

- 定义

```
typedef enum
{
    E_MI_SCL_DIRECT_BUF_OSD_PIXEL_ALPHA = 0,
    E_MI_SCL_DIRECT_BUF_OSD_CONSTANT_ALPHT
} MI_SCL_DirectOsdAlphaMode_e;
```

- 成员

成员名称	描述
E_MI_SCL_DIRECT_BUF_OSD_PIXEL_ALPHA	Osd显示每个pixel对应的alpha效果，例如argb1555/argb4444/arb8888/i2/i4/i8这些格式都能支持pixel alpha，rgb565则不会生效。
E_MI_SCL_DIRECT_BUF_OSD_CONSTANT_ALPHT	Osd硬件会忽略图像格式中的alpha位，使用统一的值设定alpha值。例如rgb565能设定其透明度

- ※ 注意事项

无

- 相关数据类型及接口

[MI_SCL_StretchBufOsd](#)

3.14. MI_SCL_DirectOsdAlphaAttr_t

- 说明

定义 Osd Alpha 属性的结构体。

- 定义

```
typedef struct MI_SCL_DirectOsdAlphaAttr_s
{
    MI_SCL_DirectOsdAlphaMode_e    eAlphaMode;
    MI_SCL_DirectOsdAlphaModePara_u stAlphaPara;
} MI_SCL_DirectOsdAlphaAttr_t;
```

- 成员

成员名称	描述
eAlphaMode	Osd Alpha的使用模式。

stAlphaPara	模式对应的参数。
-------------	----------

※ 注意事项
无

➤ 相关数据类型及接口
[MI_SCL.StretchBufOsd](#)

3.15. MI_SCL_DirectOsdInfo_t

➤ 说明
SCL 直接操作 Osdbuffer 参数信息。

➤ 定义

```
typedef struct MI_SCL_DirectOsdInfo_s
{
    MI_U32          u32X;
    MI_U32          u32Y;
    MI_U32          u32Width;
    MI_U32          u32Height;
    MI_PHY          phyAddr;
    MI_U32          u32Stride;
    MI_SCL_DirectOsdAlphaAttr_t stOsdAlphaAttr;
} MI_SCL_DirectOsdInfo_t;
```

➤ 成员

成员名称	描述
u32X	横坐标
u32Y	纵坐标
u32Width	有效像素宽度
u32Height	有效像素高度
phyAddr	Buffer 物理地址
u32Stride	Buffer 一行占字节数
stOsdAlphaAttr	Osd Alpha 属性

※ 注意事项
无

➤ 相关数据类型及接口
[MI_SCL.StretchBufOsd](#)

3.16. MI_SCL_DirectOsdBuf_t

➤ 说明
SCL 直接操作 Osdbuffer 参数类型。

➤ 定义

```
typedef struct MI_SCL_DirectOsdBuf_s
{
    MI_U8          u8OsdBufCnt;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SCL_DirectOsdInfo_t astOsdBuf[MI_SCL_DIRECT_OSD_CNT_MAX];
} MI_SCL_DirectOsdBuf_t;
```

➤ 成员

成员名称	描述
u8OsdBufCnt	OSD 图像数据的数量
ePixelFormat	OSD Buffer 像素格式
astOsdBuf	OSD 图像数据

※ 注意事项

- u8OsdBufCnt 有效区间 [0, MI_SCL_DIRECT_OSD_CNT_MAX]
#define MI_SCL_DIRECT_OSD_CNT_MAX (8)
- ePixelFormat 支持范围请参考 MI_RGN_API_V3 中 MI_RGN_AttachToChn 章节

➤ 相关数据类型及接口

[MI_SCL_StretchBufOsd](#)

4. SCL 错误码

视频输入 API 错误码如下表所示。

表 4-3: SCL API 错误码

错误代码	宏定义	描述
0XA0222000	MI_ERR_SCL_INVALID_DEVID	设备号无效
0XA0222001	MI_ERR_SCL_INVALID_CHNID	通道号无效
0XA0222002	MI_ERR_SCL_INVALID_PORTID	端口号无效
0XA0222003	MI_ERR_SCL_ILLEGAL_PARAM	参数设置无效
0XA0222004	MI_ERR_SCL_EXIST	设备已经存在
0XA0222005	MI_ERR_SCL_UNEXIST	设备已经不存在
0XA0222006	MI_ERR_SCL_NULL_PTR	输入参数空指针错误
0XA0222007	MI_ERR_SCL_NOT_SUPPORT	操作不支持
0XA0222008	MI_ERR_SCL_NOT_PERM	操作不允许
0XA0222009	MI_ERR_SCL_NOMEM	分配内存失败
0XA022200a	MI_ERR_SCL_NOBUF	缓存为满
0XA022200b	MI_ERR_SCL_BUF_EMPTY	缓存为空
0XA022200c	MI_ERR_SCL_NOTREADY	系统未初始化
0XA022200d	MI_ERR_SCL_BUSY	系统忙

5. PROCFS 介绍

5.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_scl/mi_scl0
```

```
----- start dump SCL module info -----
InitCnt  chip  version  UseSclMask
  1      5      10x      3f
----- end dump SCL module info -----

----- start dump SCL Dev info -----
DevID  refcnt  source  sclmask  cmdq  IrqEn  IrqMode  Irqnum  VsyncCnt  FrameDoneCnt  DropCnt  LowLatencyIsrCnt
  0      1      00x    3f      (null)  1      1      47      0      2277      0      0, 0, 0
----- End dump SCL dev 0 info -----

----- start dump SCL CHN info -----
DevID  ChnId  start  Crop  rot  Size  pixel  Stride  Atom  Atom0
  0      0      1(  0, 0,1280, 720)  0(1280, 720)YUV422UYVY  0  0  2277
----- End dump SCL CHN info -----

----- start dump SCL CHN info -----
DevID  ChnId  PreCnt  EnqCnt  BarCnt  checkin  checkout  DeqCnt  DropCnt  Enq0TNull
  0      0      2277    2277    2277    2277    2277    2277    0      0
----- End dump SCL CHN info -----

----- start dump scl OUTPUT PORT info -----
DevID  ChnId  PortId  sclid  bindtype  Enable  Pixel  bMirr/flip  PortCrop  OutputW  OutputH  Histogram  MatchLine  Stride  GetCnt  FailCnt  FinishCnt  fps
  0      0      0      0      0      Frame  1  YUV420SP(  0, 0)  0, 0,1280, 720  1920  1080  958  0  1920  2277  0  2277  25.00
  0      0      1      1      1  ERROR  0  YUV422UYVY(  0, 0)  0, 0, 0, 0  0  0  0  0  0  0  0  0.00
  0      0      2      2      2  ERROR  0  YUV422UYVY(  0, 0)  0, 0, 0, 0  0  0  0  0  0  0  0  0.00
  0      0      3      3      3  ERROR  0  YUV422UYVY(  0, 0)  0, 0, 0, 0  0  0  0  0  0  0  0  0.00
  0      0      4      4      4  ERROR  0  YUV422UYVY(  0, 0)  0, 0, 0, 0  0  0  0  0  0  0  0  0.00
  0      0      5      5      5  ERROR  0  YUV422UYVY(  0, 0)  0, 0, 0, 0  0  0  0  0  0  0  0  0.00
----- End dump SCL OUTPUT PORT info -----
```

- 调试信息分析

记录当前 scaler 的使用状况以及相关属性、可以动态地获取到这些信息，方便调试和测试。

- 参数说明

参数		描述
module info	InitCnt	发生 connected module 次数
	chip	Chip index
	version	Chip version
	UseSclMask	模块 scaler 使用情况

参数		描述
Device Info	DevID	设备 ID
	refcnt	重复创建 dev 次数
	source	输入源
	sclmask	设备使用 Scaler id
	cmdq	Cmdq 指针
	IrqEn	IRQ 使能状况
	IrqMode	IRQ 模式
	Irqnum	IRQ ID

	VsyncCnt	ISR vsync Cnt
	FrameDoneCnt	ISR Frame Done Cnt
	DropCnt	IRQ Frame Drop or Double Vsync
	LowLatencyIsrCnt	Low Lantency Mode Match Line IRQ Cnt

参数		描述
Channel Info	DevId	Device id
	ChnId	Channel ID
	start	Channel 是否 start
	Crop	Channel Crop 位置
	Rot	Rotation 角度
	Size	Input width/height
	pixel	Input Pixel format
	Stride	Input buffer stride
	Atom	底层拿住 buffer 数量
	Atom0Cnt	释放 buffer 后底层拿不到 buffer 的次数
	PreCnt/EnqCnt/ BarCnt/checkin/ checkout/DeqCnt/ DropCnt	Callback 接口执行次数
	EnqOTNull	Enq 时 OutBuffer 为 Null 次数统计

参数		描述
Outputport Info	DevId	Device id
	ChnId	Channel ID
	PortId	Port ID
	sclid	scaler ID
	bindtype	0: Real Time mode 1: Ring mode 2: frame mode
	Enable	使能状态
	pixel	Output pixel format
	bMirr/flip	Port 水平/竖直 翻转
	PortCrop	Port 裁剪范围
	OutputW/OutputH	输出宽/输出高
	Histogram	Scl 画面复杂度

	MatchLine	Low Lantency Mode Match Line Cnt
	Stride	Output buffer stride, 一行像素字节
	GetCnt	试获取 outputbuffer 的数量
	FailCnt	获取 outputbuffer 失败数量
	FinishCnt	处理完 output buffer 数量
	fps	Output port frame rate

5.2. echo

echo help > /proc/mi_modules/mi_scl/mi_scl0

```
-----MI_SYS_Proc_DevCommon_help_end-----
debugmutex    [/];
setprecrop    [chnid, portid, X, Y, Width, Height];
stopchnl      [chnid, ON/OFF];
dumptaskfile  [chnid, Cnt, /path/, bOnlyDumpResChange, bdumpport];
setatom       [AtomValue];
clearbuf      [chnid, portid, bClearInput, bClearOutput, bClearDoneBuff];
debuglv       [level](2:flow,4:check irq done)
               [chnid][level] (1:check buffer,2:check loop,4:check frame pts,8:check fence done,16:check func time,32:check sidebandmsg)
enableport    [chnid, portid, bEn];
protectport   [chnid portid ben clientid];
               clientid:scl(0,1,2):0x19, 0x1c, 0x1d
clearbindq    [chnid bclear];
fpsth         [chnid portid fpsint fpsfloat];
irqmode       [source IrqMode];
               source:0:isp realtime, 1:rdma0, 2:yuv realtime, 3:rdma1, 4:rdma rot
               Irqmode: 0x1:realtime, 0x2:framemode, 0x10:onlydoneint, 0x20:multiint
outputparam   [pixel chnid portid pixelmode];
outputparam   [compress chnid portid compressmode];
outputparam   [mirrorflip chnid portid bmirror bflip];
outputparam   [crop chnid portid cropX cropY cropW cropH];
outputparam   [size chnid portid sizeW sizeH];
outputparam   [all chnid portid pixelmode compressmode bmirror bflip cropX cropY cropW cropH sizeW sizeH];
crcmode       [chnid portid crcmode](0:not used, 1: used);
bytewrite     [portid offsetbyte];
patengen      [chnid bEn patengenmode];
               bEn: 0:disable 1:enable
               patengenmode: 0:static 1:dynamic
```

Echo help 查看可用命令

功能	打印所有 channel 锁的使用 function 和 line 和 time
命令	echo debugmutex > /proc/mi_modules/mi_scl/mi_scl0
参数说明	无

举例	无
----	---

功能	设置每一个 scl port 的 crop 参数，该设置直接设置给 driver, 不经过 MI 流程
命令	echo setprecrop [chnid portid X Y Width Height] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	Portid: port id
	X,Y,width,height: crop 范围
举例	echo setprecrop 0 0 0 0 800 200 > /proc/mi_modules/mi_scl/mi_scl0

功能	等待底层 buffer 处理完之后，关掉 channel，不再 trigger 底层。
命令	echo stopchnl [chnid, ON/OFF] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	ON/OFF: on 打开 stopchannel 功能，也就是 stop 掉， OFF 关闭 stopchannel 功能，打开 channel
举例	echo stopchnl 0 ON > /proc/mi_modules/mi_scl/mi_scl0
	echo stopchnl 0 OFF > /proc/mi_modules/mi_scl/mi_scl0

功能	Dump 选定的 channel 的 output 数据，并保存在/path 路径下
命令	echo dumptaskfile [chnid, Cnt, /path/, bOnlyDumpResChange, bdumpport] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	Cnt: dump 数量
	Path: 存放 dump 出来文件的路径
	bOnlyDumpResChange: 有时候有些花图在切分辨率时产生，敲 dump 命令和切分辨率操作不好配合，可以先设置该参数，然后再切分辨率，当程序识别到分辨率变化时就会将分辨率变化后的那张 buffer dump 出来。（非必选参数）
	Bdumpport: 只 dump 某一个 portid, 默认是 dump 所有在输出的 port。（非必选参数）
举例	echo dumptaskfile 0 2 /tmp > /proc/mi_modules/mi_scl/mi_scl0

功能	设置 pass 的 atomvalue
命令	echo setatom [AtomValue] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	AtomValue: Driver 最大持有 buffer 数量
举例	echo setatom 3 > /proc/mi_modules/mi_scl/mi_scl0
	ps: 有时候底层卡住，可以尝试增大一下这个 atom，可以再塞一张 buffer 给 driver，重新 trigger 一次，看是否可以恢复，或者掉帧的时候，加大是否可以满帧。

功能	清除指定 channel port 的 input/output/done buffer
----	--

命令	echo clearbuf [chnid, portid bClearInput, bclearoutput, bcleardonebuff] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	Portid: port id
	bClearInput: 清除 input 的 buffer
	bclearoutput: 清除 output 的 buffer
	bcleardonebuff: 清除 done 的 buffer
举例	echo clearbuf 0 0 1 0 0 > /proc/mi_modules/mi_scl/mi_scl0

功能	设置 debug level
命令	echo debuglv [level] > /proc/mi_modules/mi_scl/mi_scl0 echo debuglv [chnid][level] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	Level: (1) 单个 level 参数时: 2:flow, 4:check irq done (2) 两个参数时: 1:check buffer, 2:check loop, 4:check frame pts, 8:check fence done, 16:check func time, 32:check sidebandmsg
举例	echo debuglv 4 > /proc/mi_modules/mi_scl/mi_scl0 //打开 check irq done 相关的 MI 打印 echo debuglv 0 3 > /proc/mi_modules/mi_scl/mi_scl0 //打开 channel0 level3 的打印, 打印 buffer info 和 loop info

功能	设置 port 是否 enable
命令	echo enableport [chnid, portid, bEn] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	Portid: port id
	bEn: 1/0
举例	echo enableport 0 0 1 > /proc/mi_modules/mi_scl/mi_scl0

功能	保护 outputport buffer
命令	echo protectport [chnid portid ben clientid] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	Portid: port id
	ben: 使能
	Clientid: scl(0,1,2): 0x19, 0x1c, 0x1d
举例	echo protectport 0 0 1 0x19 > /proc/mi_modules/mi_scl/mi_scl0 //保护 channel0 port0 的 scl0

功能	将 Bind Q 中缓存的 buffer 消耗掉
命令	echo clearbindq [chnid bclear] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	bclear: 是否 clear
举例	echo clearbindq 0 1 > /proc/mi_modules/mi_scl/mi_scl0 //clear channel0 bindQ 有时候前端拿不到 buffer, 发现是 scl BindQ 塞满, 判断 scl 是否拿的不够快, 将 bindQ 清空, 看 BindQ 中是否还会累加上来, 如果还累加说明 scl 拿不够快, 如果不再增加, 说明是之前一次异常塞在里面一直没有消耗掉。

功能	设置打印帧率, 低于该值打印出来
命令	echo fpsth [chnid portid fpsint fpsfloat] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Chnid: channel id
	Portid: port id
	fpsint: 帧率整数位
	fpsfloat: 帧率小数位
举例	echo fpsth 0 0 30 30 > /proc/mi_modules/mi_scl/mi_scl0 // channel0 port0 中小于 30.30 的 fps 打印出来

功能	设置输入源的 irq 模式
命令	echo irqmode [source irqmode] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Source: 0:isp realtime, 1:rdma0, 2:yuv realtime, 3:rdma1, 4:rdma rot
	Irqmode: 0x1:realtime, 0x2:frame mode, 0x10:onlydoneint, 0x20:multiint
举例	echo irqmode 0 1 > /proc/mi_modules/mi_scl/mi_scl0

功能	设置 output port 参数
命令	echo outputparam pixel [chnid, portid pixelmode] > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam compress [chnid, portid compressmode] > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam mirrorflip [chnid, portid bmirror bflip] > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam crop [chnid, portid cropX cropY cropW cropH] > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam size [chnid, portid sizeW sizeH] > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam all [chnid, portid pixelmode compressmode bmirror bflip

	cropX cropY cropW cropH sizeW sizeH] > /proc/mi_modules/mi_scl/mi_scl1
参数说明	Chnid: channel id
	Portid: port id
	Pixelmode: 输出的 pixel 类型, 填入 MI_SYS_PixelFormat_e 中对应的数值
	Compressmode: 输出的压缩模式, 填入 MI_SYS_CompressMode_e 中对应的数值
	Bmirror: 输出的使能水平翻转
	Bflip: 输出的使能垂直翻转
	cropX: 输出的端口裁剪 X 坐标
	cropY: 输出的端口裁剪 Y 坐标
	cropW: 输出的端口裁剪宽度
	cropH: 输出的端口裁剪高度
	sizeW: 输出的端口宽度
	sizeH: 输出的端口高度
举例	echo outputparam pixel 0 0 11 > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam compress 0 0 5 > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam mirrorflip 0 0 1 1 > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam crop 0 0 0 0 1920 1080 > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam size 0 0 1920 1080 > /proc/mi_modules/mi_scl/mi_scl1
	echo outputparam all 0 0 11 5 1 1 0 0 1920 1080 1920 1080 > /proc/mi_modules/mi_scl/mi_scl1

功能	设置输出端口的 CRC 模式
命令	echo crcmode [chnid portid crcmode] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	chnid: channel id
	Portid: port id
	crcmode: 0:禁用, 1: 使用
举例	echo crcmode 0 1 1 > /proc/mi_modules/mi_scl/mi_scl0

功能	设置输出端口的字节偏移
命令	echo bytewrite [portid offsetbyte] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	Portid: port id
	offsetbyte: 字节偏移量

举例	echo crcmode 0 1 1 > /proc/mi_modules/mi_scl/mi_scl0
----	--

功能	设置输入源的测试图形
命令	echo patengen [chnid bEn patengenmode] > /proc/mi_modules/mi_scl/mi_scl0
参数说明	chnid: channel id
	bEn: 0:禁用, 1: 使用
	patengenmode: 0:静态测试图形 1: 动态测试图形
举例	echo patengenmode 0 1 0 > /proc/mi_modules/mi_scl/mi_scl0